

ClickHouse

Este apartado está dedicado a la **integración de Kafka con ClickHouse** (más información en [Integrating Kafka with ClickHouse](#)). Como describe la documentación oficial, existen diferentes mecanismos pero en este tutorial se va a utilizar concretamente el [Kafka Table Engine](#)

1. KSQL

Se supone la existencia de un *topic* **weblog** que contiene mensajes obtenidos del log de un servidor *Apache*. Se muestra a continuación el resultado de la orden `print weblog from beginning limit 1;`

```
Key format: `\_(\ツ)\_/\` - no data processed
Value format: AVRO
rowtime: 2025/04/24 07:49:18.098 Z,
key: <null>,
value: {"GEOIP": {"IP": "5.188.62.140", "as": {"ORGANIZATION": {"NAME": "Petersburg Internet Netw
ork ltd."}, "NUMBER": 34665}, "GEO": {"POSTAL_CODE": null, "CITY_NAME": null,
"COUNTRY_NAME": "Russia", "REGION_ISO_CODE": null, "TIMEZONE": "Euro
pe/Moscow", "LOCATION": {"LON": 37.6068, "LAT": 55.7386}, "COUNTRY_ISO_CODE":
"RU", "REGION_NAME": null, "CONTINENT_CODE": "EU"}}, "URL": "/admini
strator/index.php", "HTTP": {"RESPONSE": {"BODY": {"BYTES": 4494},
"STATUS_CODE": 200}, "REQUEST": {"METHOD": "POST", "REFERRER": null},
"VERSION":
: "1.1"}, "USER_AGENT": {"UAID": "1a184c9ea2806985af981dea8ffe25e9ce454f46",
"ORIGINAL": "Mozilla/5.0 (Windows NT 6.2) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/41.0.2227.1 Safari/537.36", "OS": {"VERSION": "8", "NAME":
"Windows", "full": "Windows 8"}, "VERSION": "41.0.2227.1", "NAME": "
Chrome", "DEVICE": {"NAME": "Other"}}, "APACHE_TS": 1640998294010},
partition: 0
```

Sobre dicho tópico se crea un *stream* de acuerdo con el siguiente código:

```
CREATE STREAM `weblog` (
  geoip STRUCT<ip VARCHAR,
    `as` STRUCT<organization STRUCT<name VARCHAR>,
      number INTEGER>,
  geo STRUCT<postal_code VARCHAR,
    city_name VARCHAR,
    country_name VARCHAR,
    region_iso_code VARCHAR,
    timezone VARCHAR,
    location STRUCT<lon DOUBLE,
      lat DOUBLE>>
```

```

country_iso_code VARCHAR,
region_name VARCHAR,
continent_code VARCHAR>
>,
url VARCHAR,
http STRUCT<response STRUCT<body STRUCT<bytes INTEGER>,
status_code INTEGER>,
request STRUCT<method VARCHAR,
referrer VARCHAR>,
version VARCHAR
>,
user_agent STRUCT<uaid VARCHAR,
original VARCHAR,
os STRUCT<version VARCHAR,
name VARCHAR,
`full` VARCHAR>,
version VARCHAR,
name VARCHAR,
device STRUCT<name VARCHAR>>>,
apache_ts TIMESTAMP
)
WITH (
KAFKA_TOPIC='weblog',
VALUE_FORMAT='AVRO');

```

De cara a controlar el flujo de información desde *Kafka* a *ClickHouse* se define un segundo *stream* denominado *raw_ips* que se alimenta invocando un *insert* desde *weblog*

```

-- Stream de "llave de paso"
CREATE STREAM `raw_ips` (
  geoip STRUCT<ip VARCHAR,
    `as` STRUCT<organization STRUCT<name VARCHAR>,
    number INTEGER>,
  geo STRUCT<postal_code VARCHAR,
    city_name VARCHAR,
    country_name VARCHAR,
    region_iso_code VARCHAR,
    timezone VARCHAR,
    location STRUCT<lon DOUBLE,
    lat DOUBLE>,
    country_iso_code VARCHAR,
    region_name VARCHAR,
    continent_code VARCHAR>
  >,
  apache_ts TIMESTAMP
)
WITH (kafka_topic='raw_ips', key_format='KAFKA', value_format='AVRO',
PARTITIONS=1, REPLICAS=1);

-- Tabla demo. Crear topic
-- Se carga de manera controlada por el stream raw_ips
CREATE TABLE demoips
WITH (kafka_topic='demoips', key_format='JSON', value_format='AVRO')
AS SELECT

```

```

    struct(ip := geoip->ip,
           nomorg := geoip->`as`->organization->name,
           numorg := geoip->`as`->number,
           postal_code := geoip->geo->postal_code,
           city_name := geoip->geo->city_name,
           country_name := geoip->geo->country_name,
           country_iso_code := geoip->geo->country_iso_code,
           region_name := geoip->geo->region_name,
           region_iso_code := geoip->geo->region_iso_code,
           continent_code := geoip->geo->continent_code,
           tz := geoip->geo->timezone,
           lat := geoip->geo->location->lat,
           lon := geoip->geo->location->lon) as geoip,
    COUNT(apache_ts) as N
FROM `raw_ips`
GROUP BY struct(ip := geoip->ip,
               nomorg := geoip->`as`->organization->name,
               numorg := geoip->`as`->number,
               postal_code := geoip->geo->postal_code,
               city_name := geoip->geo->city_name,
               country_name := geoip->geo->country_name,
               country_iso_code := geoip->geo->country_iso_code,
               region_name := geoip->geo->region_name,
               region_iso_code := geoip->geo->region_iso_code,
               continent_code := geoip->geo->continent_code,
               tz := geoip->geo->timezone,
               lat := geoip->geo->location->lat,
               lon := geoip->geo->location->lon);

SELECT geoip, N as ipc
FROM demoips;

--
-- Insertar datos de test
INSERT INTO `raw_ips`
SELECT geoip,
       apache_ts
FROM `weblog`
WHERE geoip->ip IS NOT NULL AND user_agent->uaid IS NOT NULL
and geoip->ip like '130.%'
;

```

2. ClickHouse

El proceso de integración requiere los siguientes elementos:

- una tabla donde se van a cargar los datos, en este caso se denomina *geoips*
- una tabla con **engine Kafka** que captura los datos
- una vista que materialice la transformación realizando automáticamente las conversiones necesarias cada vez que se reciban mensajes en el *topic* de *Kafka*, en este caso en *demoips*

```

-- Tabla destino de los datos
DROP TABLE ibd00.geoips;
CREATE TABLE ibd00.geoips
(
    ip IPv4 NOT NULL,
    nomorg Nullable(String),
    numorg Nullable(Int32),
    postal_code Nullable(String),
    city_name Nullable(String),
    country_name Nullable(String),
    country_iso_code Nullable(String),
    region_name Nullable(String),
    region_iso_code Nullable(String),
    continent_code Nullable(String),
    tz Nullable(String),
    lat Nullable(Float64),
    lon Nullable(Float64),
    ipc Int32
) ENGINE = MergeTree ORDER BY (ip);

-- Tabla de integración
DROP TABLE ibd00.kafka_demoips;
CREATE TABLE ibd00.kafka_demoips
(
    N Int32
) ENGINE = Kafka() SETTINGS
    kafka_broker_list = 'kafka:9092',
    kafka_topic_list = 'demoips',
    kafka_group_name = 'clickhouse',
    format_avro_schema_registry_url='http://schema-registry:8081',
    kafka_format = 'AvroConfluent';

-- Vista de materialización de la transformación
DROP TABLE ibd00.geoips_mv;
CREATE MATERIALIZED VIEW ibd00.geoips_mv TO ibd00.geoips AS
SELECT toIPv4(simpleJSONExtractString(_key, 'IP')) as ip,
       coalesce(NULLIF(simpleJSONExtractString(_key, 'NOMORG'), ''),
               'NOMORG_IS_NULL') as nomorg,
       simpleJSONExtractUInt(_key, 'NUMORG') as numorg,
       coalesce(NULLIF(simpleJSONExtractString(_key, 'POSTAL_CODE'), ''),
               'POSTAL_CODE_IS_NULL') as postal_code,
       coalesce(NULLIF(simpleJSONExtractString(_key, 'CITY_NAME'), ''),
               'CITY_NAME_IS_NULL') as city_name,
       coalesce(NULLIF(simpleJSONExtractString(_key, 'COUNTRY_NAME'), ''),
               'COUNTRY_NAME_IS_NULL') as country_name,
       coalesce(NULLIF(simpleJSONExtractString(_key, 'COUNTRY_ISO_CODE'),
               ''), 'COUNTRY_ISO_CODE_IS_NULL') as country_iso_code,
       coalesce(NULLIF(simpleJSONExtractString(_key, 'REGION_NAME'), ''),
               'REGION_NAME_IS_NULL') as region_name,
       coalesce(NULLIF(simpleJSONExtractString(_key, 'REGION_ISO_CODE'),
               ''), 'REGION_ISO_CODE_IS_NULL') as region_iso_code,
       coalesce(NULLIF(simpleJSONExtractString(_key, 'CONTINENT_CODE'),
               ''), 'CONTINENT_CODE_IS_NULL') as continent_code,
       coalesce(NULLIF(simpleJSONExtractString(_key, 'TZ'), ''),
               'TZ_IS_NULL') as tz,
       simpleJSONExtractFloat(_key, 'LAT') as lat,
       simpleJSONExtractFloat(_key, 'LON') as lon,

```

```
      N as ipc  
FROM ibd00.kafka_demoips;  
  
select * from ibd00.geoips g ;
```